

Observability

Building agents by building their harness

Lesson 5 of 10 · Production

Where we left off

Lesson 4 finished the harness.

Five primitives. 48 lines.

It works.

A new kind of lesson

Lessons 1 to 4 each added a **primitive**.

Lessons 5 to 10 add **no new primitive**.

They **harden** the five we have.

Production, not a sixth primitive

Each production lesson takes one concern.

It shows where that concern lives among the five.

Then it adds the smallest code that handles it.

The question for today

The agent ran for a minute and gave a wrong answer.

What happened?

Or a simpler one

The agent finished.

What did that cost?

What you can see today

The final answer.

That is all.

The steps in between are gone.

What was in between

- every model call
- every command it asked for
- how long each took
- how many tokens each used

All of it happened. None of it was kept.

Observability

Recording what the harness did, so you can read it afterwards.

Traces, logs, cost.

Where does it live?

Observability is **built on control flow**.

Why control flow

Control flow is the one place that sees the **whole sequence**.

Every model call and every tool run passes through the loop.

So the loop is where you watch.

Not the other four

The model interface only knows one call.

Output only knows one tool.

Only the loop sees call, tool, call, tool, stop.

The worked example

`production/05-observability/quark.py`

Lesson 4's `quark.py` plus one function.

Line count

Lesson 4: 48 lines.

Lesson 5: **59 lines.**

The new function

```
trace(**event)
```

It appends one JSON line to `.quark/traces.jsonl`.

Why JSON lines

One event per line.

Append only. No parsing the whole file to add one.

Readable by `jq`, `grep`, or a ten-line script.

Three new imports

```
import json, time, uuid
```

- `json` writes the line
- `time` measures and stamps
- `uuid` names the run

Every event carries two fields

- `ts` : when it happened
- `run` : a run id

Why a run id

`.quark/traces.jsonl` collects every run.

The id lets you pull out **one** run.

Without it, runs blur together.

Four kinds of event

- `start`
- `model`
- `tool`
- `too_long`

Event: start

The run began.

Marks where one run's lines begin.

Event: model

One call to the model.

- `seconds`
- `stop_reason`
- `input_tokens`
- `output_tokens`
- `cache_read`
- `cache_write`

Why stop_reason

Lesson 1: `end_turn` or `max_tokens` .

Lesson 2: `tool_use` .

Now you can see **why each call stopped**.

Why the cache fields

Lesson 4 marked the system prompt for caching.

`cache_read` and `cache_write` show whether it **worked**.

Event: tool

One command the model asked for.

- `cmd`
- `seconds`
- `exit`
- `chars`

Why chars, not output

The trace records **how much** came back.

Not the output itself.

Traces stay small. The conversation already holds the text.

Event: too_long

Lesson 4's reactive compaction.

The API said the prompt was too long.

Now that moment is **on the record**.

Where `trace()` is called

Inside the loop.

Around the model call.

Around the tool run.

Timing

Take `time.time()` before.

Take it again after.

The difference is `seconds` .

What did not change

Control flow still decides everything.

`trace()` only **writes down** what it decided.

Run it

```
uv run production/05-observability/quark.py \  
    "how many lines are in README.md?"
```

Same agent as Lesson 4.

Read the trace

```
tail -n 5 .quark/traces.jsonl | jq .
```

`jq` pretty-prints each line.

What to look for

- how many `model` events
- which one had the tool call
- how long each `tool` took
- how the tokens grew

Tokens grow every step

Each request carries everything so far.

`input_tokens` rises with each model event.

You can **see** the growing `working_memory` .

One run out of many

Filter by run id:

```
jq 'select(.run == "<id>")' .quark/traces.jsonl
```

Going further

`quark.py` is one way to do it.

`observability.py` is a richer one.

observability.py

Built on Lesson 3's `control_flow.py` .

Same loop, with **spans** and **cost**.

From events to spans

An event is a point in time.

A **span** has a start, an end, and a **parent**.

The span tree

```
run
├── model
├── tool
├── model
└── tool
```

Model and tool spans have the run as their parent.

span()

A context manager, `with span(...)` .

It times the block and writes the record when the block ends.

Errors are recorded

If the block raises, the span **records the error**.

Then it **re-raises** it.

Observing must not hide a failure.

Cost

`cost()` turns token counts into dollars.

It uses a `PRICE` dict.

The prices are examples

The numbers in `PRICE` are **example rates**.

The file says so.

Look up the current price before you trust the totals.

Live step lines

While it runs, it prints a line per step.

You watch it **as it happens**, not afterwards.

A summary row

`row()` prints one line at the end.

Status, calls, tools, problems, time, tokens in and out, cached, cost.

The answer to "what did that cost, and did anything fail?"

Report mode

```
uv run production/05-observability/observability.py report
```

Reads the log. Runs nothing.

A separate log

Spans go to `.quark/spans.jsonl`.

`quark.py` writes `.quark/traces.jsonl`.

Different file. Different format. Don't mix them up.

What observability can be

- **what** it records: events, spans, tokens, cost
- **where** it goes: a file, a service
- **when** you read it: live or after
- **how much** it keeps: counts or full text

Tracing products

Services exist that collect and display this.

They are this layer, packaged.

You can hand it off. You can also write 11 lines.

What to take away

Rule: observability records what the harness did, from the one place that sees the whole sequence: control flow.

It adds no primitive.

Notice what observability never does

It never decides what runs, or when to stop.

It reads the clock, `usage` and exit codes, and changes none of them.

It only **watches**.

What's missing

You can now see everything the agent does.

You can see it run any command it likes.

Nothing stops it.

Next: Guardrails

Lesson 6 stops it before it does harm.